

BrokerVerse

Out-of-the-box insurance broking platform

Solution Architecture · Document 04 of 11

Technology Stack

Product	BrokerVerse
Document	04 Technology Stack
Version	1.0
Date	29 September 2026
Prepared by	iorta TechNXT
Classification	Confidential: for the client, iorta TechNXT and authorised partners

Document control

Document	BrokerVerse Solution Architecture: 04 Technology Stack
Document ID	BV-SA-04
Version	1.0
Status	Issued for review
Date	29 September 2026
Prepared by	iorta TechNXT
Source baseline	BrokerVerse source repository, branch brokerverse-platform (backend/, brokerverse/, docker-compose.yml, render.yaml, docs/)

Version history

Version	Date	Author	Change
1.0	29 September 2026	iorta TechNXT	Initial issue

Review and approval

Role	Name	Signature	Date
Solution architect			
DevOps / infrastructure lead			
Information security			
Business owner			

Conventions

- Statements of fact are derived from the code, configuration and database of the baseline above (file paths are given relative to the repository). Figures measured on the local test system are labelled **test data** or **indicative**.
- Targets, sizes, retention periods and thresholds that are not facts of the code are labelled **Recommended / to be confirmed by the business and DevOps**.
- Items marked **Gap** are defects or missing capabilities found while preparing this document.

Contents

- 1 Introduction..... 4
 - 1.1 Purpose and scope..... 4
 - 1.2 Stack overview..... 4
- 2 Backend (backend/)..... 5
 - 2.1 Runtime dependencies..... 5
 - 2.2 Development and test dependencies..... 5
 - 2.3 Scripts and tools..... 5
- 3 Front end (brokerverse/)..... 7
 - 3.1 Runtime dependencies..... 7
 - 3.2 Build and delivery..... 7
- 4 Infrastructure and platform..... 9
- 5 Lifecycle and risk notes..... 10
- 6 Related documents..... 11
- 7 Glossary..... 12

1 Introduction

1.1 Purpose and scope

This document lists the technologies used by BrokerVerse, with the versions resolved in the lock files of the baseline, their purpose and their licence type, together with the runtime, infrastructure, build and test tooling. It is the reference for dependency management, security patching and licence review.

Sources: `backend/package.json` and `backend/package-lock.json`, `brokerverse/package.json` and `brokerverse/package-lock.json` (licences read from the installed packages), `backend/Dockerfile`, `brokerverse/Dockerfile`, `brokerverse/nginx.conf`, `docker-compose.yml`, `render.yaml`, `brokerverse/.github/workflows/deploy.yml` and `docs/GO_LIVE_CHECKLIST.md`. Versions are the exact versions in the lock files (the manifests use caret ranges; `npm ci` installs the locked versions).

1.2 Stack overview

Layer	Technology
Browser	Modern evergreen browsers (build targets <code>>0.2%</code> , <code>not dead</code> , <code>not op_mini all</code>)
Front end	React 18.2 single-page application, PrimeReact 10 / PrimeFlex / PrimiIcons, Redux Toolkit 2, React Router 6, Formik, i18next, axios, Chart.js, FullCalendar, Sass; built with Create React App 5 (react-scripts) and craco 7
Web serving	Amazon S3 + CloudFront (target); nginx 1.27-alpine (container option); Render static site (blueprint)
API	Node.js 22 (ES modules), Express 4.22, node-postgres 8, zod 3, pino 9, helmet 8, jsonwebtoken 9, bcryptjs 2, multer 1.4, nodemailer 10, node-cron 4
Database	PostgreSQL 16 (local 16.13; <code>postgres:16-alpine</code> in Docker Compose; Amazon RDS for PostgreSQL 16 target) with <code>pgcrypto</code>
File storage	Local file system / persistent volume (<code>UPLOAD_DIR</code>); Amazon EFS target
E-mail	Any SMTP server through <code>SMTP_URL</code> (for example Amazon SES)
Containers	Docker images <code>node:22-alpine</code> (API), <code>node:22-alpine</code> build stage + <code>nginx:1.27-alpine</code> (web); Docker Compose
CI/CD	GitHub Actions (front end: build, S3 sync, CloudFront invalidation); backend pipeline to be set up
Cloud	AWS ap-southeast-1 (Singapore): S3, CloudFront, and for the backend ALB, ECS Fargate / App Runner / EC2, RDS, EFS, Secrets Manager / SSM, CloudWatch (recommended)

2 Backend (backend/)

2.1 Runtime dependencies

Package	Version	Licence	Purpose in BrokerVerse
express	4.22.3	MIT	HTTP server, routing, middleware chain (<code>src/app.js</code> , module routers)
pg (node-postgres)	8.23.0	MIT	PostgreSQL client and connection pool (<code>src/db/pool.js</code> , max 10 connections), type parsers for numeric, bigint, date
zod	3.25.76	MIT	Request validation schemas (<code>lib/validate.js</code>)
pino	9.14.0	MIT	Structured JSON logging with redaction
pino-http	10.5.0	MIT	HTTP access log with request id
helmet	8.3.0	MIT	Security headers
cors	2.8.6	MIT	Cross-origin policy from <code>CORS_ORIGINS</code>
jsonwebtoken	9.0.3	MIT	Access, refresh, challenge and approval tokens (HS256)
bcryptjs	2.4.3	MIT	Password hashing (pure JavaScript bcrypt, cost 10)
multer	1.4.5-lts.2	MIT	Multipart uploads held in memory with size and count limits
nodemailer	10.0.12	MIT-0	SMTP delivery of the e-mail outbox
node-cron	4.6.0	ISC	Cron scheduling of the scheduled jobs
dotenv	16.6.1	BSD-2-Clause	Loads <code>.env</code> in development

The backend has 13 direct runtime dependencies (141 packages including transitive ones, per the code review). The following are implemented in the code base instead of third-party libraries: PDF writers (`src/tools/pdf.js`, `modules/documents/pdf.js`), XLSX writer (`tools/xlsx.js`) and ZIP container (`tools/zip.js` over `node:zlib`), CSV with formula-injection guarding (`tools/csv.js`), TOTP two-factor (`lib/totp.js`), AES-256-GCM secret encryption and HMAC link signing (`lib/secrets.js` over `node:crypto`), rate limiting (`lib/rateLimit.js`), and workbook import with inflate caps (`lib/uploadLimits.js`).

2.2 Development and test dependencies

Package	Version	Licence	Purpose
vitest	2.1.9	MIT	Unit and API tests (<code>backend/test</code> , 40 files, about 345 test cases)
supertest	7.3.0	MIT	HTTP assertions against the in-process app
eslint, @eslint/js, globals	9.39.5	MIT	Linting (<code>npm run lint</code> ; strict rules including <code>no-console</code> , <code>eql</code>)

2.3 Scripts and tools

Command	Purpose
<code>npm start</code>	<code>node src/server.js</code> : production configuration check, migrations, seed, HTTP server, scheduler
<code>npm run dev</code>	Development server with <code>node --watch</code>
<code>npm run migrate</code> , <code>npm run seed</code> , <code>npm run db:reset</code>	Apply migrations; run the seed; drop and rebuild the schema (development only)
<code>npm run export:api</code>	Generate OpenAPI, Postman and Excel API documentation from the route registry

Command	Purpose
npm run purge:sample	Remove sample / demo data before go-live (dry run by default)
npm test, npm run lint	Tests and lint

3 Front end (brokerverse/)

3.1 Runtime dependencies

Package	Version	Licence	Purpose
react, react-dom	18.2.0	MIT	UI library
react-scripts (Create React App)	5.0.1	MIT	Build tool chain (webpack, Babel, Jest, ESLint)
@craco/craco	7.1.0	Apache-2.0	Overrides the CRA webpack configuration (<code>craco.config.js</code>)
primereact	10.3.1	MIT	Component library (tables, forms, dialogs, toasts, calendars)
primeflex	3.3.1	MIT	CSS utility classes
primeicons	6.0.1	MIT	Icon font
@reduxjs/toolkit	2.0.1	MIT	Application state (<code>src/redux/store.js</code>)
react-redux	9.0.4	MIT	React bindings for Redux
react-router, react-router-dom	6.30.6	MIT	Client-side routing (<code>routes/MainRoute.js</code>)
history	5.3.0	MIT	Navigation history helper
formik	2.4.5	Apache-2.0	Forms and validation
axios	1.20.0	MIT	HTTP client (interceptor with token refresh)
i18next, react-i18next, i18next-browser-languagedetector	25.8.13 / 16.5.4 / 8.2.1	MIT	Internationalisation (English, Thai)
chart.js	4.5.1	MIT	Dashboard charts
@fullcalendar/core, daygrid, react	6.1.x	MIT	Calendar views (agent events)
moment	2.29.4	MIT	Date handling in older screens
js-cookie	3.0.8	MIT	Cookie access (legacy)
react-pro-sidebar	1.1.0-alpha.1	MIT	Side menu
sass	1.69.5	MIT	SCSS styles and theme
@fontsource/nunito	5.3.0	OFL-1.1	Nunito web font, bundled (no external font CDN)
redux-logger	3.0.6	MIT	Redux logging in development builds only
web-vitals	2.1.4	Apache-2.0	Performance metrics hook

The `@testing-library/*` packages are listed under dependencies but are only used by tests.

3.2 Build and delivery

Item	Detail
Build	<code>npm run build</code> (<code>craco build</code>); <code>REACT_APP_BASE_URL</code> (the API base, for example <code>https://<brokerverse-url>/api</code>) is compiled in at build time; <code>GENERATE_SOURCEMAP=false</code> in the Docker and Render builds
Pipeline	<code>.github/workflows/deploy.yml</code> : on push to dev, Node 20, <code>npm install --legacy-peer-deps</code> , build with <code>CI=false</code> , <code>aws-actions/configure-aws-credentials@v4</code> (access keys in secrets), <code>aws s3 sync build/ s3://\$S3_BUCKET --delete</code> , <code>aws cloudfront create-invalidation --paths "/"</code>

Item	Detail
Container	Multi-stage Dockerfile: node:22-alpine build, nginx:1.27-alpine runtime with nginx.conf (SPA fallback, /static/ cached for one year, index.html no-cache, /api/ proxied to the API, client_max_body_size 25m, security headers)

4 Infrastructure and platform

Component	Version / service	Purpose and status
Node.js	22 (image node:22-alpine; local v22.22.2)	API runtime. engines in package.json says >=20; the front-end pipeline builds with Node 20
PostgreSQL	16	System of record; RDS target (document 07, 09)
nginx	1.27-alpine	Web container of the Docker Compose option
Docker / Docker Compose	Compose file format without version key	Local and single-server deployment
Render	Blueprint render.yaml	Managed demo / UAT hosting option
GitHub Actions	actions/checkout@v4, setup-node@v4, configure-aws-credentials@v4	Front-end CI/CD
AWS	ap-southeast-1	S3 and CloudFront in use for the front end; backend services per the go-live checklist (ALB, ECS / App Runner / EC2, RDS, EFS, Secrets Manager or SSM)
Monitoring	CloudWatch (recommended)	Logs, metrics, alarms (document 11)

5 Lifecycle and risk notes

Item	Observation (from docs/review/CODE_REVIEW.md and the lock files)
Backend advisories	<code>npm audit</code> reported 0 backend advisories after nodemailer 10 and node-cron 4 upgrades (29 Sep 2026).
Front-end advisories	71 advisories remain (3 critical, 35 high), almost all build-time transitive dependencies of react-scripts 5.0.1; react-router 6 and craco have moderate advisories. Create React App is no longer maintained; a migration to Vite (or another maintained tool chain) and React Router 7 is the planned remediation.
Node.js versions	Align the front-end pipeline (Node 20) and the backend image (Node 22); Node 20 reaches end of life on 30 April 2026 per the Node.js release schedule, so move the pipeline to Node 22 LTS (recommended).
Pipeline credentials	The workflow uses long-lived AWS access keys stored as GitHub secrets. Recommended: GitHub OIDC with an IAM role limited to the bucket and distribution.
Pipeline tests	The test and SonarQube steps of the workflow are commented out; the workflow edits a <code>tsconfig.json</code> that the project does not have (<code>harmless, true</code>). Recommended: run <code>npm test</code> and the backend tests in CI.
Licences	All listed runtime dependencies use permissive licences (MIT, MIT-0, ISC, BSD-2-Clause, Apache-2.0) and the Nunito font uses the SIL Open Font License; no copyleft licence was found among the direct dependencies. A full transitive licence scan (for example license-checker) is recommended before release.

6 Related documents

This document is one of eleven that together form the BrokerVerse Solution Architecture. They are maintained and issued together (source and build: [docs/architecture/README.md](#)).

No.	Document	File
01	Solution Component Diagram and Application Architecture	01_BrokerVerse_Solution_Component_Diagram_Application_Architecture.pdf
02	Database Design	02_BrokerVerse_Database_Design.pdf
03	Database Inventory	03_BrokerVerse_Database_Inventory.pdf
04	Technology Stack (this document)	04_BrokerVerse_Technology_Stack.pdf
05	Shared Service Components	05_BrokerVerse_Shared_Service_Components.pdf
06	Capacity and Performance	06_BrokerVerse_Capacity_and_Performance.pdf
07	High Availability and Resiliency	07_BrokerVerse_High_Availability_and_Resiliency.pdf
08	RTO and RPO	08_BrokerVerse_RTO_and_RPO.pdf
09	Backup and Recovery	09_BrokerVerse_Backup_and_Recovery.pdf
10	Data Archival, Housekeeping and Restoration	10_BrokerVerse_Data_Archival_Housekeeping_and_Restoration.pdf
11	Monitoring	11_BrokerVerse_Monitoring.pdf

7 Glossary

Terms and abbreviations used in this document.

Term	Meaning
ALB	Application Load Balancer (AWS): distributes HTTPS requests over the API containers and checks their health.
API	Application programming interface; here the BrokerVerse REST API under /api served by the Node.js backend.
bcrypt	Adaptive password-hashing function (bcryptjs library) used for user passwords.
CloudFront	AWS content delivery network that serves the single-page application and can route /api to the backend.
CloudWatch	AWS monitoring service: logs, metrics, dashboards and alarms.
CORS	Cross-Origin Resource Sharing: browser rule set that the API restricts to the configured web origins (CORS_ORIGINS).
CRA / craco	Create React App (react-scripts) build tool chain, customised with craco (Create React App Configuration Override).
Cron	Time-based schedule expression (minute hour day month weekday) used by the scheduled jobs.
ECS / Fargate	Elastic Container Service (AWS) and its serverless compute mode for running the API containers.
EFS	Elastic File System (AWS): shared, multi-AZ network file system proposed for the upload directory.
HMAC	Keyed-hash message authentication code; signs file links and hashes one-time codes.
HS256	HMAC-SHA256 JSON Web Token signature algorithm, the only one the API accepts.
Migration	Numbered SQL file in backend/src/db/migrations applied once, in order, on API start.
nginx	Web server used in the web container to serve the SPA and forward /api to the API container.
RDS	Amazon Relational Database Service, the managed PostgreSQL target.
RPO	Recovery point objective: the maximum acceptable data loss, measured in time.
RTO	Recovery time objective: the maximum acceptable time to restore service.
S3	Amazon Simple Storage Service: object storage hosting the SPA build (and proposed archive store).
Seed	Idempotent data load run on every API start: reference data always, sample data only when SEED_SAMPLE_DATA is on.
SES	Amazon Simple Email Service, a possible SMTP relay.
SMTP	Simple Mail Transfer Protocol; the API sends e-mail through the server in SMTP_URL.
SPA	Single-page application: the React front end served as static files.
TOTP / 2FA	Time-based one-time password (RFC 6238) used for two-factor authentication.
UAT	User acceptance testing environment.
zod	TypeScript-first schema validation library used to validate request bodies and parameters.